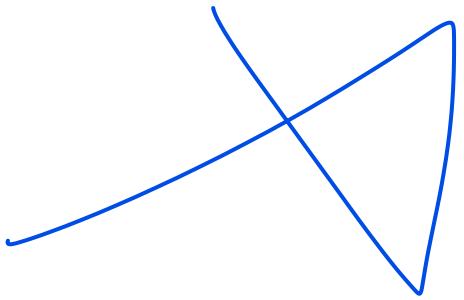
Tree

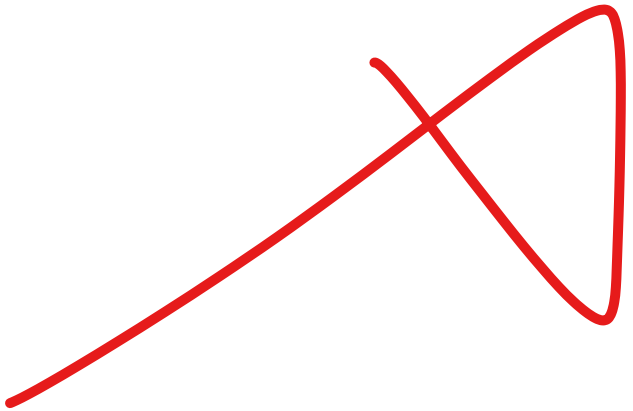


TREE - PART 1

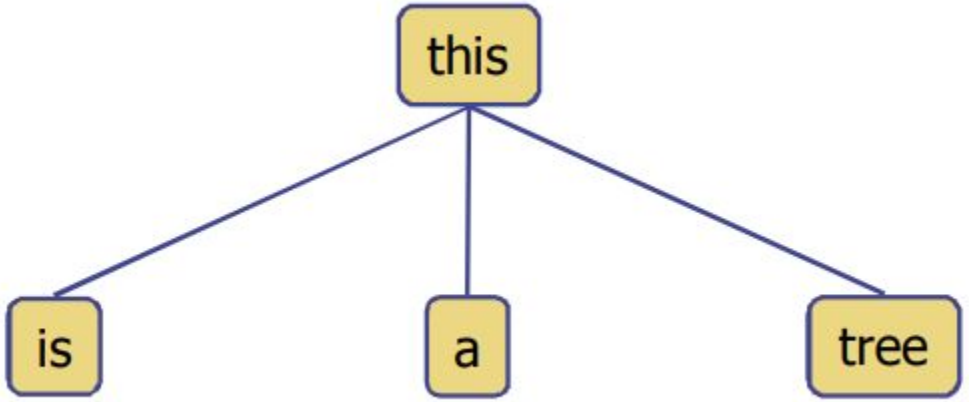
Definition

Terminologies

Depth and Height



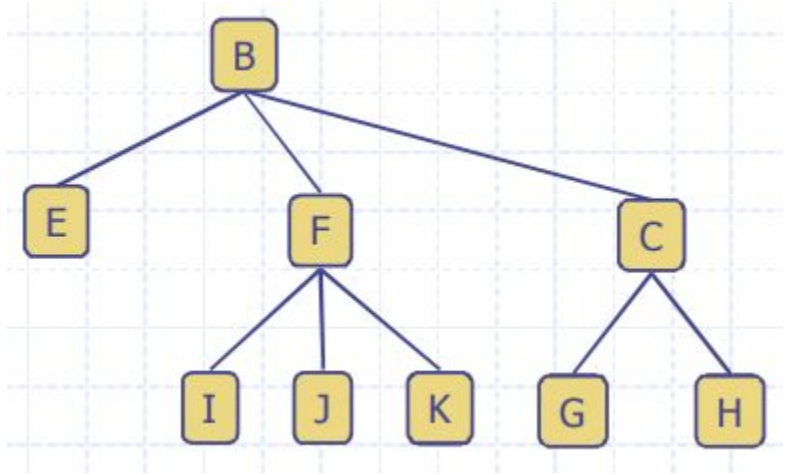
What is a Tree?



- A **connected acyclic** graph is a tree.
- In computer science, a tree is an abstract model of a hierarchical structure.
- A tree consists of nodes with a parent-child relationship
- Applications:
  - Organizational charts
  - File systems
  - Programming environments

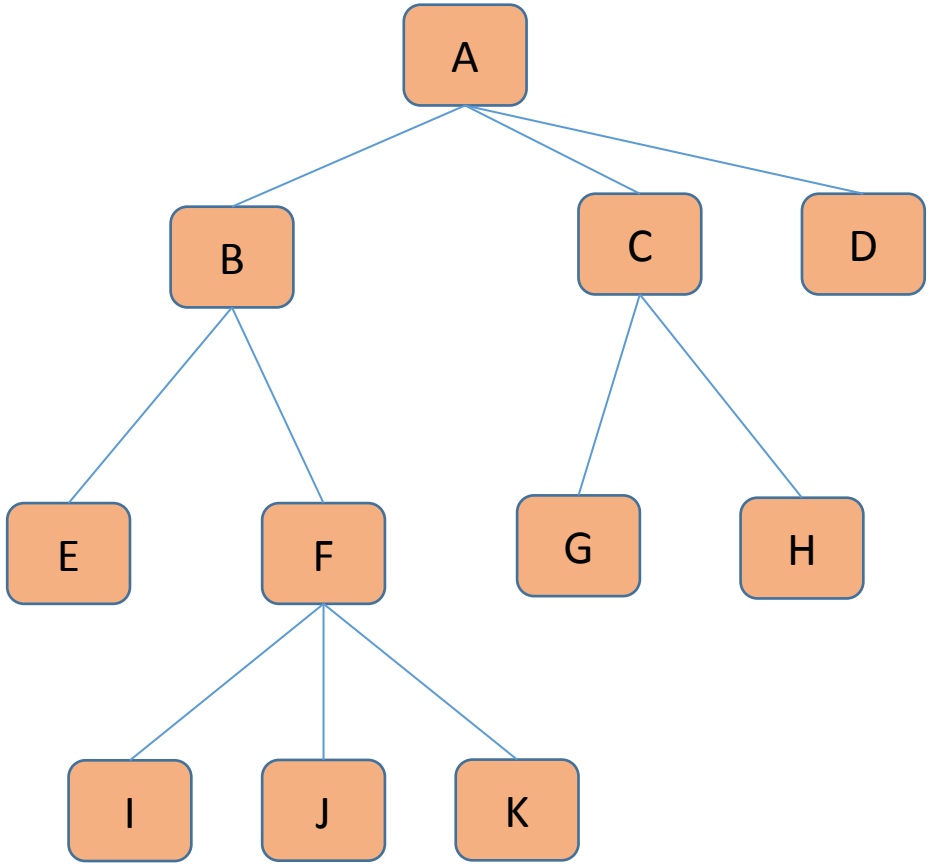
What is a Tree? (Cont.)

- A tree **T** is a set of nodes in a parent-child relationship with the following properties:
  - T has a special **node r**, called the root of T, with no parent node
  - Each node v of T, different from r, **has a parent node u**



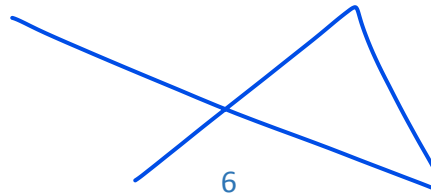
Tree Terminologies

- **Root**: node without parent (A)
- **Internal node:** node with at least one child (A, B, C, F)
- **External node (Leaf):** node without children (E, I, J, K, G, H, D)
- **Ancestors of a node:** parent, grandparent, grand-grandparent, etc.
- **Descendants of a node:** child, grandchild, grand-grandchild, etc.
- **Depth of a node:** number of ancestors, excluding the node itself
- **Height of a tree:** maximum depth of any node
- **Siblings:** two nodes that are children of the same parent



Note

- We will explore the implementation codes for Trees later on
  - We already did a bit with heaps
- For now lets us get some general idea about some tree related algorithms



Depth and Height of a Node

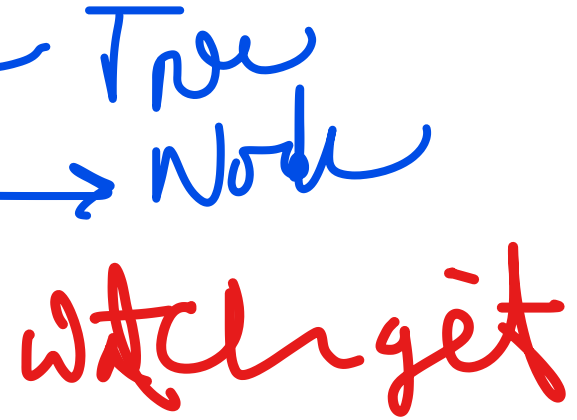
- The **depth** of a node v can be recursively defined as follows
  - If v is the root, then the depth of v is 0.
  - Otherwise, the depth of v is one plus the depth of the parent of v

```
Algorithm depth(T, v)
  if T.isRoot(v) then
    return 0
  else
    return 1 + depth(T, T.parent(v))
```

Depth and Height of a Node

- The **height** of a node v can be recursively defined as follows
  - If v is a leaf node, then the height of v is 0.
  - Otherwise, the height of v is one plus the maximum height of a child of v The height of a tree T is the height of the root of T The height of a tree T is equal to the maximum depth of a leaf node of T

```
Algorithm height(T, v)
 if T.isLeaf(v) then
     return 0
 else
     h = 0
 for each w ε T.children(v) do
     h = max(h, height(T, w))
 return 1 + h
```



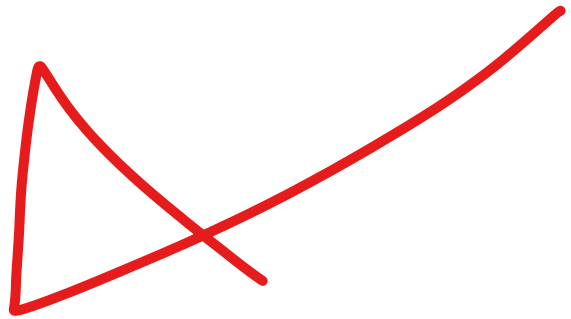
TREE - PART 2

Ordered Trees

Tree Traversal

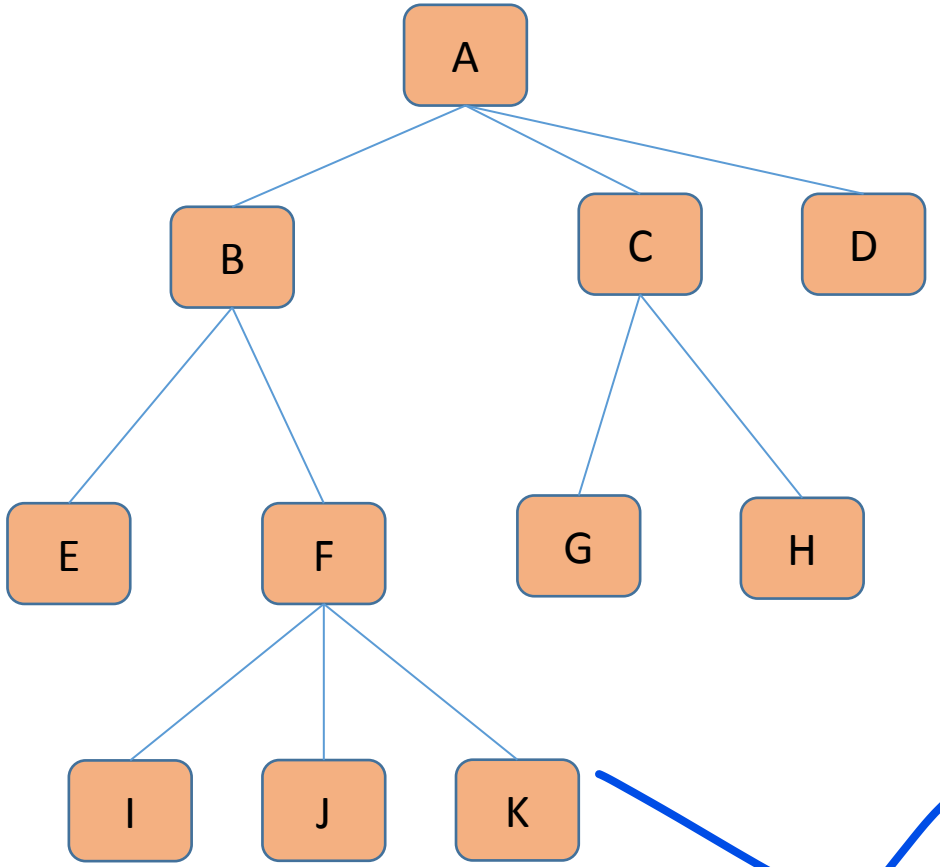
Ordered Trees

- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an ordered tree in which every node has at most two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**



Ordered Trees

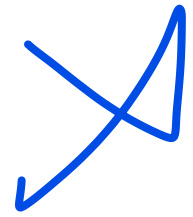
- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an ordered tree in which every node has at most two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**



What happens if it is an ordered tree?
vs
What happens if it is not an ordered tree?

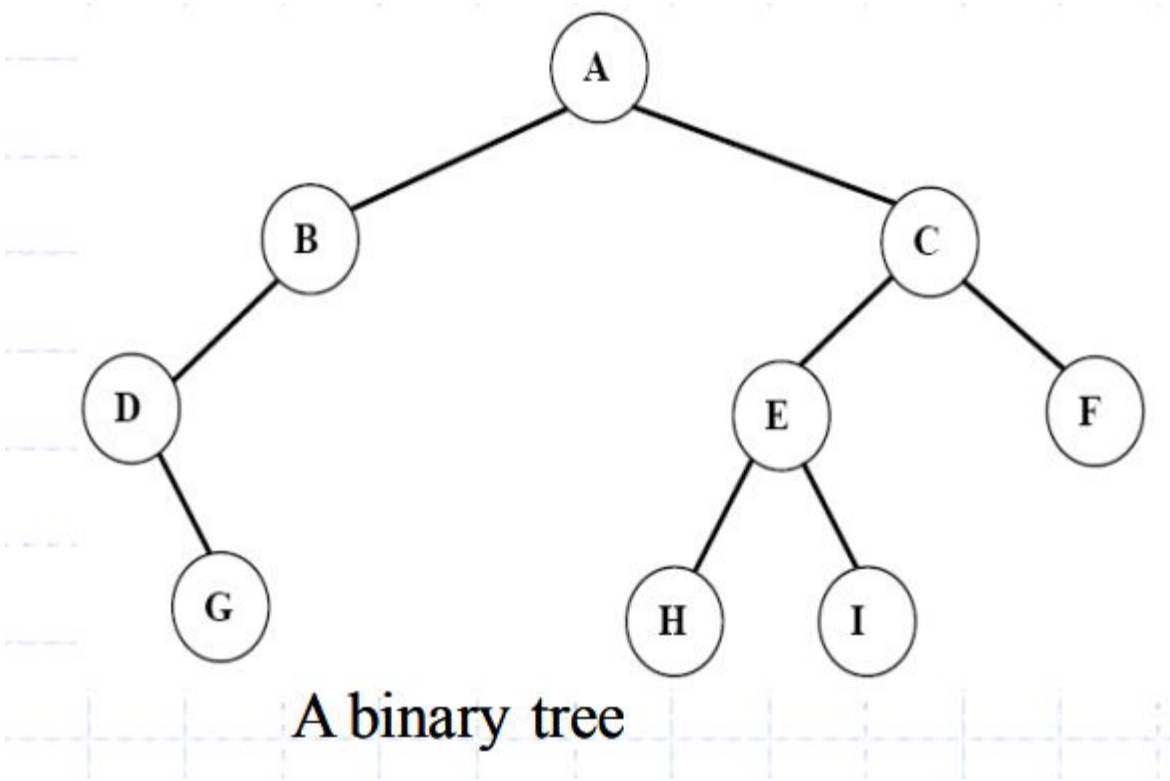
Ordered Trees

- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an **ordered** tree in which every node has **at most** two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**



Ordered Trees

- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an **ordered** tree in which every node has **at most** two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**

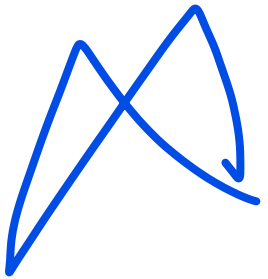


A binary tree

Recall the term m-ary tree

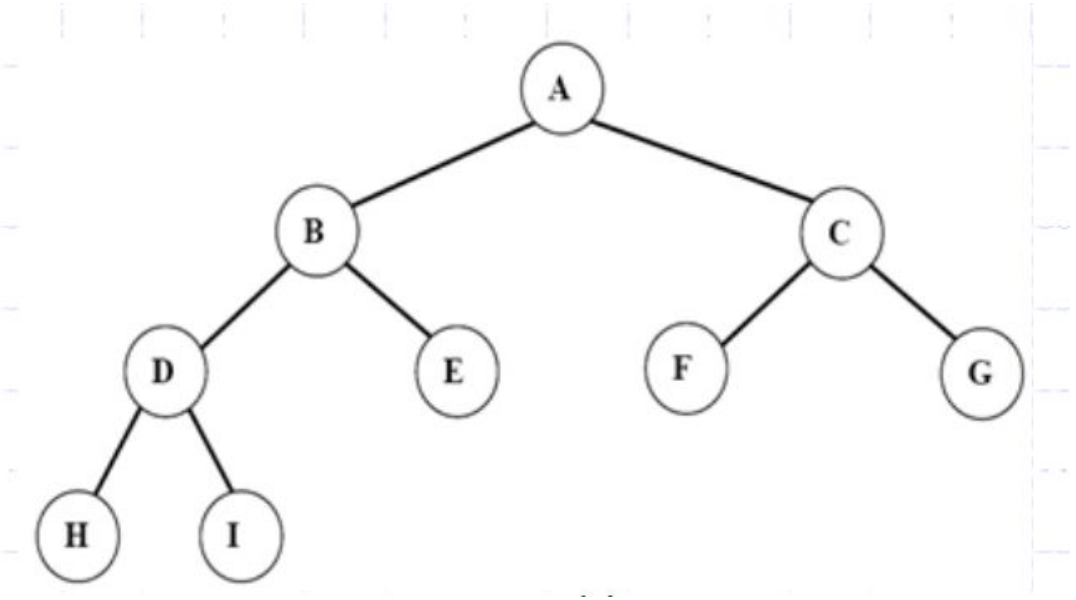
Ordered Trees

- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an **ordered** tree in which every node has **at most** two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**



Ordered Trees

- A tree is **ordered** if there is a linear ordering defined for each child of each node.
- A **binary tree** is an **ordered** tree in which every node has **at most** two children.
- If each node of a tree has either zero or two children, the tree is called a **proper binary tree.**



Recall the term proper m-ary tree

Tree Traversal

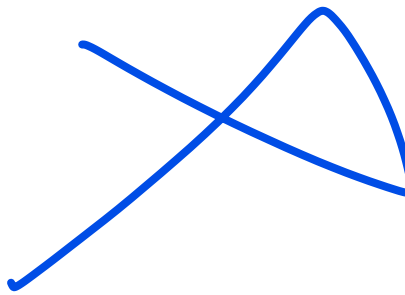
- A traversal of a tree T is a systematic way of visiting **all the nodes** of T
- Traversing a tree involves visiting the **root** and traversing its **subtrees**
- There are the following traversal methods:
  - Preorder Traversal
  - Postorder Traversal
  - Inorder Traversal

Tree Traversal

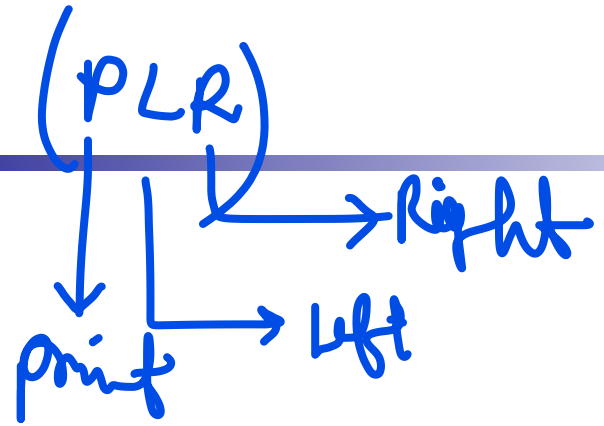
Preorder Traversal (General Tree)

- In a preorder traversal, a node is visited before its descendants.
- If a tree is ordered, then the subtrees are traversed according to the order of the children

```
Algorithm preOrder(v)
    visit(v)
    for each child w of v
        preorder (w)
```



Tree Traversal



Preorder Traversal (Binary Tree)

- Visit the root.
- Traverse the left subtree in Preorder.
- Traverse the right subtree in Preorder.

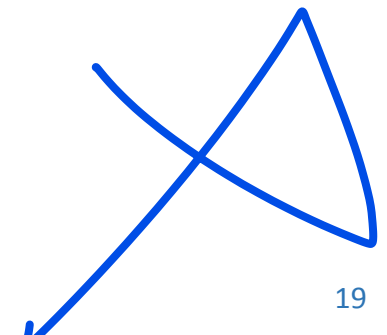
```
Algorithm preOrder(v)
    visit(v)
    for each child w of v
        preorder (w)
```

Tree Traversal

Postorder Traversal (General Tree)

- In a postorder traversal, a node is visited after its descendants

```
Algorithm postOrder(v)
    for each child w of v
        postOrder (w)
    visit(v)
```



Tree Traversal

Postorder Traversal (Binary Tree)

- Traverse the left subtree in Preorder.
- Traverse the right subtree in Preorder.
- Visit the root.

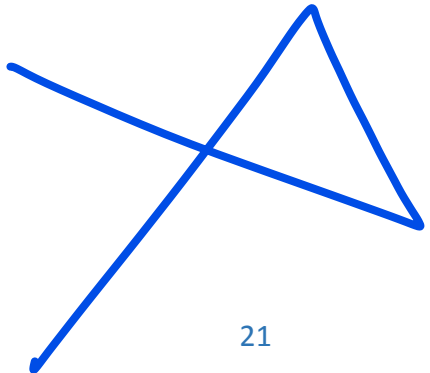
```
Algorithm postOrder(v)
    for each child w of v
        postOrder (w)
    visit(v)
```

Tree Traversal

Inorder Traversal (General Tree)

- In an inorder traversal a node is visited after its left subtree and before its right subtree

```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```



Tree Traversal

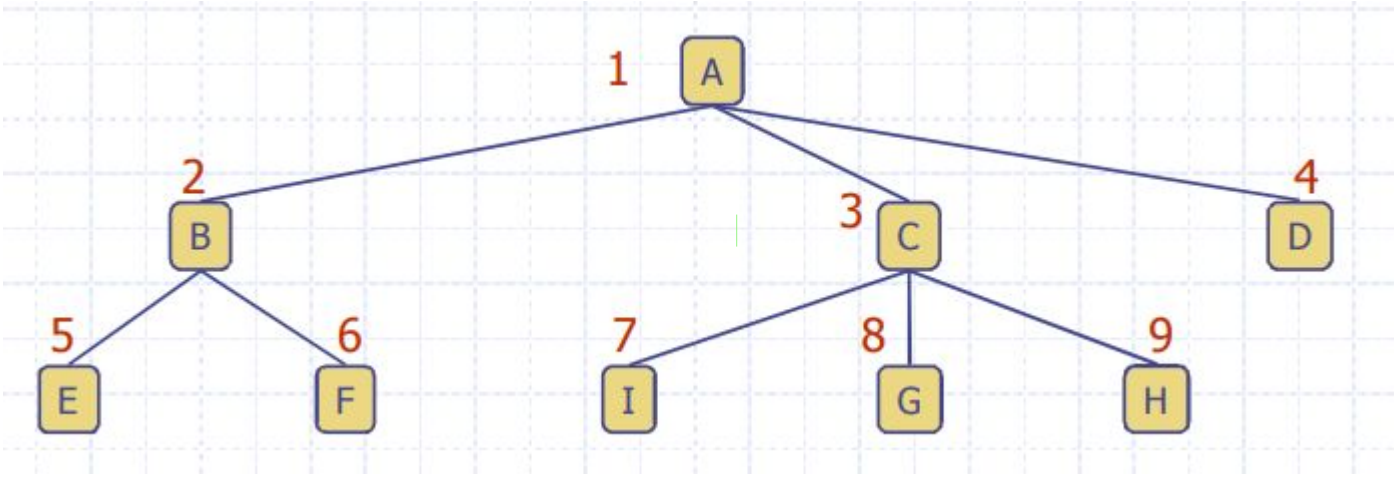
Inorder Traversal (Binary Tree)

- Traverse the left subtree in inorder.
- Visit the root.
- Traverse the right subtree in inorder.

```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```

Level Order Tree Traversal

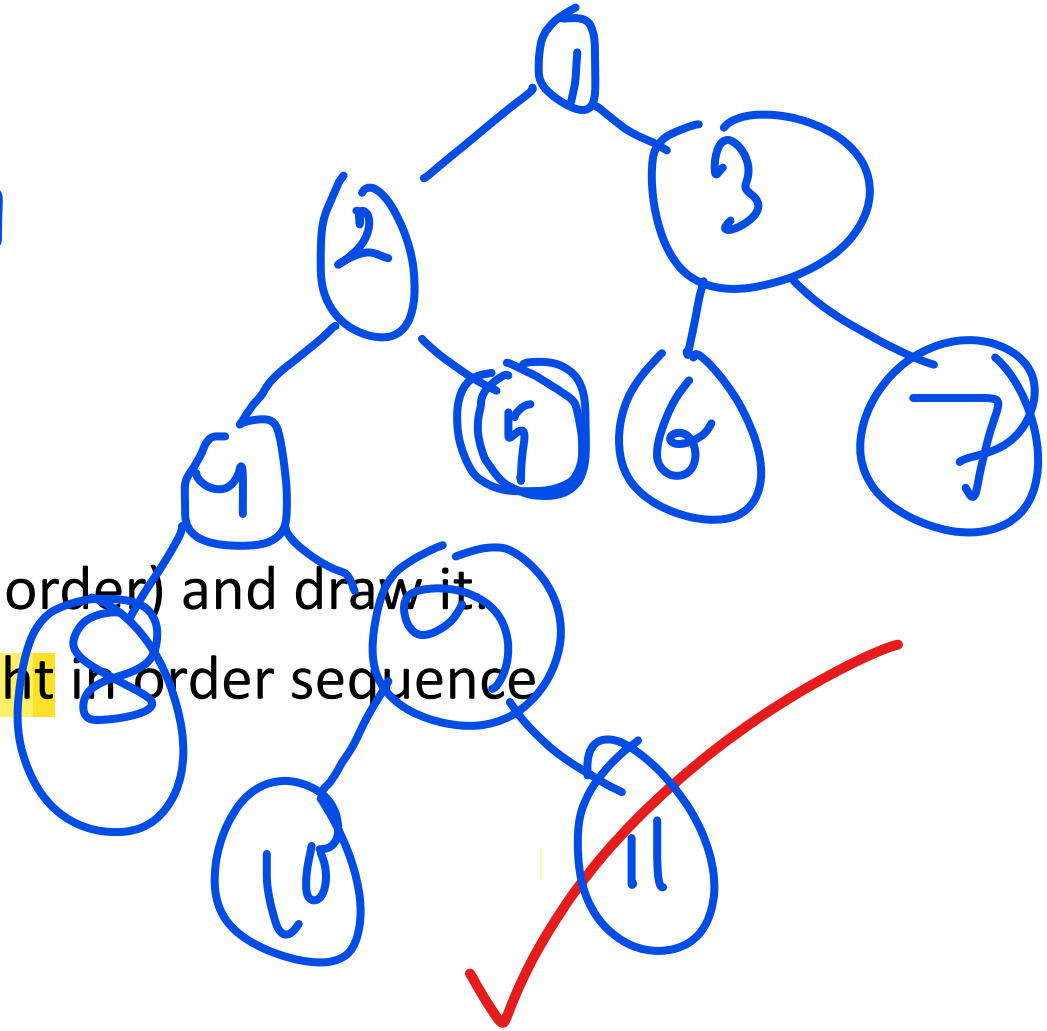
- In a level order traversal, every node on a level is visited before going to a lower level



Level order: A B C D E F I G H

Drawing Trees based on Tree Traversals

- Pre Order and Inorder
  - Pre-Order : 1, 2, 4, 8, 9, 10, 11, 5, 3, 6, 7
  - In-Order : 8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7
- Method:
  - Consider the In Order Sequence.
  - Identify the Root (first element from pre order) and draw it.
  - Determine left in order sequence and right in order sequence
  - Repeat until the tree is drawn

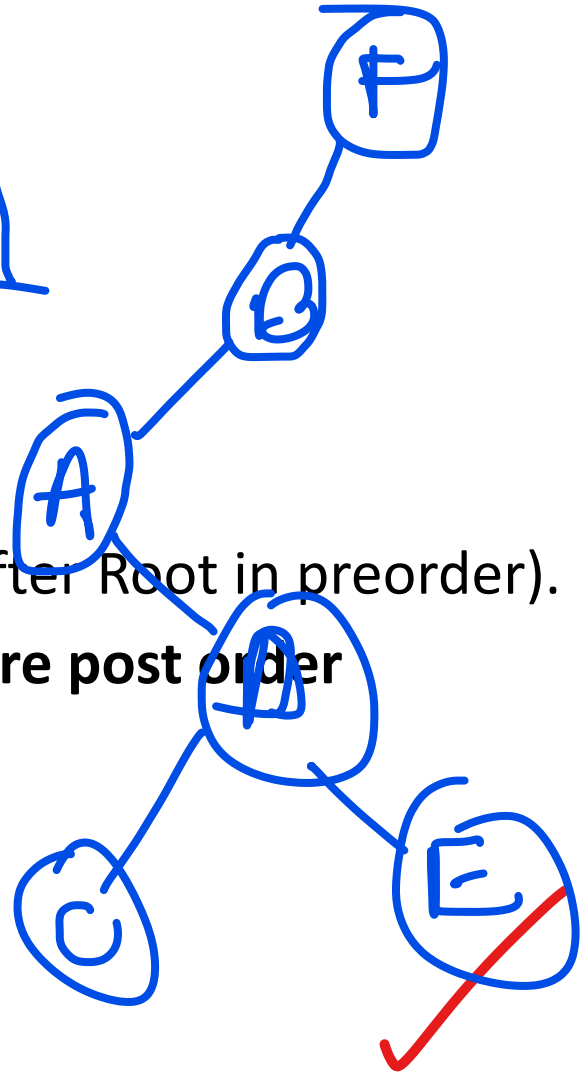


Drawing Trees based on Tree Traversals

- Post Order and Inorder
  - Pre-Order : 9, 1, 2, 12, 7, 5, 3, 11, 4, 8
  - In-Order : 9, 5, 1, 7, 2, 12, 8, 4, 3, 11
- Method:
  - Consider the In Order Sequence.
  - Identify the Root (last element from pre order) and draw it.
  - Determine left in order sequence and right in order sequence
  - Repeat until the tree is drawn

Drawing Trees based on Tree Traversals

- Pre Order and Post Order
  - Pre-Order : F, B, A, D, C, E, G, I, H (Root, Left, Right)
  - Post-Order : A, C, E, D, B, H, I, G, F (Left, Root, Right)
- Method:
  - Consider the **Post Order Sequence**.
  - Identify the **Root** (with the help of pre-order).
  - Identify the **partitioning element** (Second Element after Root in preorder).
  - **Partition** the **Post Order Sequence** to obtain **two more post order sequences**.
  - Repeat until the entire tree is drawn.



Thank You